

MODULE NAME: ENTERPRISE APPLICATION DEVELOPMENT

MODULE CODE: EAPD7111/w

NAME: TSHEPO THULO

STUDENT NUMBER: ST10535587

POE: ASSIGNMENT 1 PART 1

Table of Contents	
Summary	3
Architectural Framework Strategy: TOGAF.....	4
Phases of ADM From A through D	4
GLMS Data Flow with TOGAF ADM Phase (A-D).....	5
GLMS UML Diagram	5
Non-Functional Requirements.....	6
References	8

Summary

The Global Logistics Management System (GLMS) is designed to help TechMove's transition from manual processes to a modern, digital workflow. This is done by consolidating freight contracts, driver scheduling, invoicing, and client management into a single platform. The system does away with the need for spreadsheets, endless email threads, and phone tag. This centralization enhances information accessibility, reduces errors, and increases overall efficiency and compliance.

The system serves internal users like logistics coordinators and finance teams, external users like drivers and clients, and technical teams that maintain the platform.

Architecturally, the GLMS is built on a cloud-based microservices approach, enabling scalability, flexibility, and high availability. This design supports future growth, allows independent service deployment, and ensures the system remains reliable and responsive as business demand increases.

Architectural Framework Strategy: TOGAF

The TOGAF framework was selected because it provides a structured and comprehensive approach for designing and managing complex enterprise systems such as the Global Logistics Management System (GLMS). It is widely used in enterprise architecture to support large-scale system design and transformation initiatives (The Open Group, 2022).

Firstly, TOGAF's Architecture Development Method (ADM) offers a clear, step-by-step process for transitioning from existing manual processes to a fully integrated digital platform. This is particularly important in logistics environments where data is often fragmented across different systems and needs to be progressively unified into a single architecture (Harrison, 2011).

Secondly, TOGAF emphasises strong alignment between business objectives and IT systems. This ensures that core logistics activities such as freight contract management, driver scheduling, and invoicing are directly supported by well-designed applications and reliable data structures, improving overall operational effectiveness (Ross, Weill and Robertson, 2006).

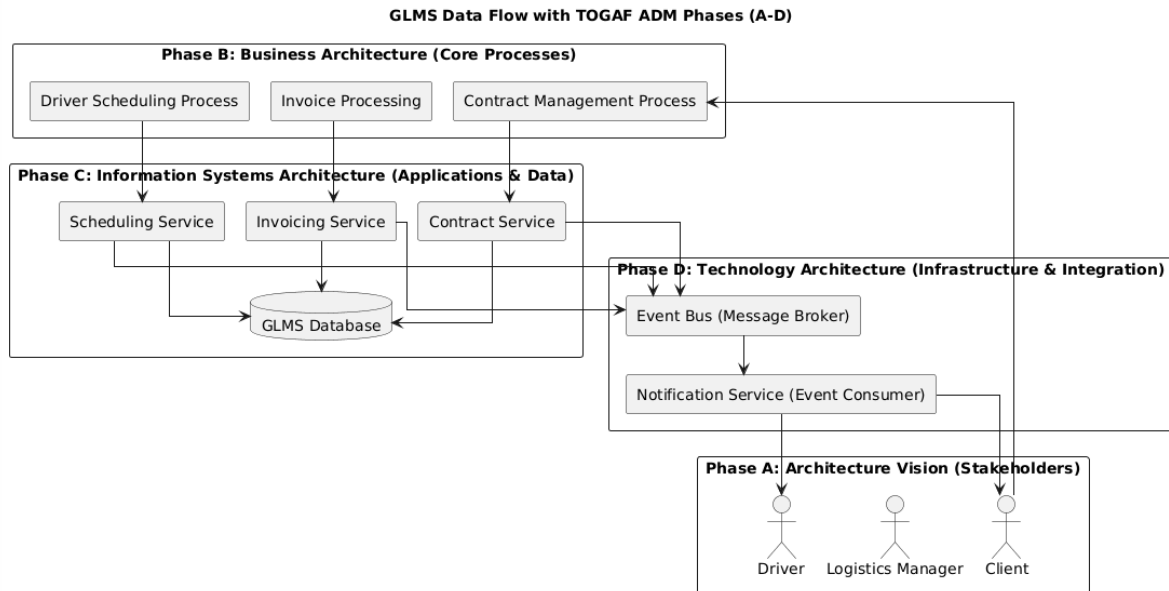
Lastly, TOGAF promotes standardisation, governance, and scalability, which are essential for a global logistics organisation like TechMove. These principles help ensure that the system can handle high transaction volumes, maintain compliance, and adapt to future business growth without compromising performance or reliability (Bernard, 2012).

Phases of ADM From A through D

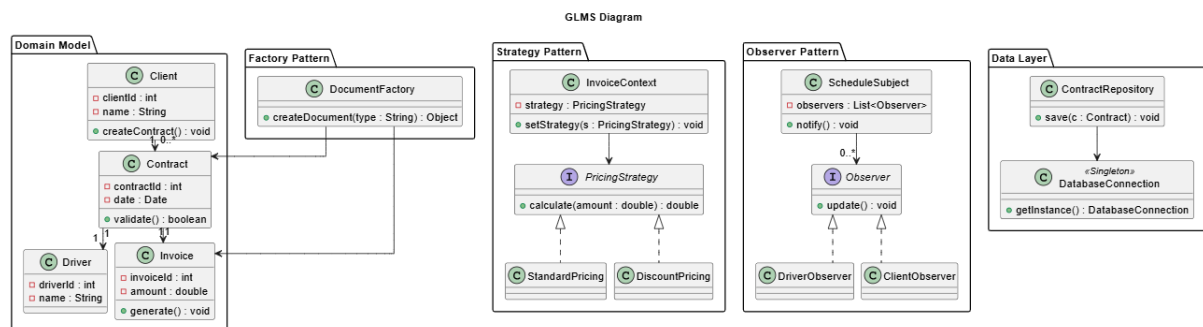
The following table maps the GLMS architecture to the TOGAF ADM phases A–D, demonstrating how the system design aligns with enterprise architecture best practices (The Open Group, 2022).

ADM Phase	Focus Area	GLMS Application
Phase A	Architecture Vision	Defines the strategic vision of a unified GLMS platform that replaces manual processes with an integrated digital solution. This improves visibility across contracts, scheduling, and finance while addressing data fragmentation and inefficiencies.
Phase B	Business Architecture	Structures core business processes including contract management, service request validation, driver scheduling, and invoicing. Business rules ensure only valid contracts are processed, while clearly defined roles (clients, drivers, logistics managers) support efficient operations.
Phase C	Data & Application Architecture	Establishes a centralised data model for contracts, clients, and service requests to ensure consistency and integrity. Defines independent services (contract, scheduling, invoicing) to support modularity and scalability. Integrates external APIs (e.g., currency conversion) to support accurate financial operations.
Phase D	Technology Architecture	Specifies the implementation environment using cloud infrastructure (Azure) to enable scalability and global access. Containerisation (Docker) and orchestration (Kubernetes) support high availability and resilience. API gateways and secure protocols ensure reliable, secure communication between services.

GLMS Data Flow with TOGAF ADM Phase (A-D)



GLMS UML Diagram



The above GLMS UML diagram represents structured domain-driven design that is supported by design patterns to improve modularity and scalability.

In the domain model, a one-to-many relationship exists between the Client and the Contract. That means one Client can create a multiple Contract.

Contract has a one-to-one relationship with both the Driver and Invoice. Object creation is handled through the **Factory pattern** where the DocumentFactory centralises the logic for Contract and Invoice.

The dependency injection happens between the InvoiceContext and the PricingStrategy. This dynamically assigns the PricingStrategy with either the StandardPricing or the DiscountPricing. This is how the **Strategy Pattern** is implemented.

In the scheduling component, the **Observer Pattern** uses an event-driven mechanism. ScheduleSubject maintains a collection of observers (0..*) and triggers updates through the notify() method. When there are scheduling changes, both DriverObserver and ClientObserver are automatically notified. This enables real-time communication without direct dependencies.

In the data layer, persistence is handled via the ContractRepository, which abstracts database operations and promotes separation of concerns. It depends on a *DatabaseConnection* implemented as a **Singleton**, ensuring a single shared instance is used across the system to optimise resource usage and maintain consistency.

Collectively, these patterns enhance decoupling by ensuring that components interact through well-defined interfaces rather than direct dependencies. Validation logic is centralised within services and factory components, ensuring consistent enforcement of business rules, such as validating contracts before processing service requests (Gamma et al., 1994; Newman, 2021).

Non-Functional Requirements

Scalability

In TOGAF, scalability is mainly addressed during the Technology Architecture phase (Phase D), where the focus is on making sure the system can handle both normal and peak business demand.

Scalability will be important because the GLMS system will need to handle sudden spikes in activity, such as 50,000 new contracts being created at once. To deal with this, there are two approaches that can be applied. The vertical scaling and horizontal scaling.

Vertical scaling means increasing the power(processors/storage) of a single server. That is not ideal with the GLMS as it has hardware limitations.

Horizontal scaling means adding more servers to the system.

The system uses a horizontally scalable architecture where multiple instances of services run in a distributed environment instead of relying on a single server. This helps spread the workload and avoids performance bottlenecks (Tanenbaum and Van Steen, 2017).

For example, when thousands of new freight contracts are uploaded, the workload for the GLMS system can be distributed across multiple application servers. One set of instances can handle the processing of contracts while the other handles the invoicing.

The system also uses Kubernetes to manage containerised services, allowing it to automatically scale the number of service instances up or down depending on demand (Burns et al., 2016). On top of this, cloud autoscaling ensures that extra computing resources are added during peak periods and removed when demand decreases, improving both performance and cost efficiency (Amazon Web Services, 2024).

To further improve performance, round robin load balancing technique is used to distribute traffic evenly across multiple service instances. For example, when clients submit freight contracts or

drivers access schedules simultaneously, requests are sequentially routed across available instances. That ensures no single instance becomes overloaded and maintaining stable response times (NGINX, Inc., 2023). **Caching** is also used to store frequently accessed data such as contracts and driver schedules in memory, that helps with reducing database load and improves response speed (Redis Ltd., 2023).

Message queues are used to handle large volumes of requests asynchronously, ensuring that contract submissions are processed without overwhelming the system (Newman, 2021).

Overall, this setup ensures that the GLMS can scale effectively during high demand while still maintaining good performance and reliability.

Availability and Resilience

Availability is briefly touched on above but to add on to that, the GLMS is designed to remain available even when parts of the system fail. This is achieved by running multiple instances of services in containers, so if one instance goes down, others can continue handling requests. This will aid in improving the reliability of the system and reduce downtime (Microsoft, 2023).

Security

The system deals with sensitive logistics data and financial data, so security is critical. To ensure that only authorised users can access specific parts of the system, Role-based Access Control (RBAC) is used. All communication between services is secured using HTTPS, and sensitive data is encrypted both in transit and at rest.

An API gateway is used to enforce security rules such as request validation and rate limiting. In addition, container security practices like image scanning and runtime protection are applied within Kubernetes to reduce vulnerabilities (Microsoft, 2023).

References

- The Open Group (2022) *TOGAF® Standard, 10th Edition*. Available at: <https://www.opengroup.org/togaf> (Accessed: 09 April 2026).
- Bernard, S.A. (2012) *An Introduction to Enterprise Architecture*. 3rd ed. Bloomington: AuthorHouse.
- Harrison, R. (2011) *TOGAF 9 Foundation Study Guide*. Van Haren Publishing.
- Ross, J.W., Weill, P. and Robertson, D.C. (2006) *Enterprise Architecture as Strategy: Creating a Foundation for Business Execution*. Boston: Harvard Business School Press.
- Burns, B., Grant, B., Oppenheimer, D., Brewer, E. and Wilkes, J. (2016) 'Borg, Omega, and Kubernetes', *ACM Queue*, 14(1), pp. 70–93.
- Newman, S. (2021) *Building Microservices*. 2nd edn. Sebastopol: O'Reilly Media.
- Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.
- Microsoft (2023) *Security in microservices*. Available at: <https://learn.microsoft.com/en-us/azure/architecture/microservices/design/security> (Accessed: 11 April 2026).
- Redis Ltd. (2023) *What is Redis?*. Available at: <https://redis.io/docs/about/> (Accessed: 11 April 2026).
- Microsoft (2023) *Availability patterns*. Available at: <https://learn.microsoft.com/en-us/azure/architecture/framework/resiliency/overview> (Accessed: 11 April 2026).
- Amazon Web Services (2024) *Auto Scaling Groups Documentation*. Available at: <https://docs.aws.amazon.com/autoscaling/> (Accessed: 11 April 2026).
- NGINX, Inc. (2023) *What is load balancing?*. Available at: <https://www.nginx.com/resources/glossary/load-balancing/> (Accessed: 11 April 2026).